AULA 16

A clínica do Nav2, e a sessão que se reproduz sem o robô

Etapa 8 · terça, 03/11/2026 · sala SJ205 · Prof. Dácio Moreira de Souza

Um sistema que só funciona com você na frente dele não está entregue.

É isso que o item 2 do TP4 cobra ao pedir uma árvore de comportamento E uma sessão gravada que se reproduz offline: a árvore decide sem você, e o bag prova sem você.

Cenas dos últimos episódios: onde paramos

- O Nav2 subiu com o mapa de vocês, e a nuvem fechou depois do 2D Pose Estimate
- Antes disso: três nuvens de 6 a 8 cm, e uma delas 5,92 m fora do lugar
- A frase que ficou: concordância interna não é evidência
- E a aula começou com um defeito meu: rviz:=true, catorze processos, nenhum RViz2
- O launch_arguments do include não é escopado — e nada reclamou

O que você sai sabendo hoje

- Descobrir, em dez segundos, por que o SEU Nav2 não está navegando
- Mandar o robô até um objetivo e provar que ele chegou — não que ele planejou
- Escolher, e saber dizer, ONDE mora a decisão de interromper a navegação
- Medir uma sessão gravada com o robô desligado: a análise que não depende da bancada

E o que cada uma vale no SEU projeto

- Diagnosticar: você deixa de depender de mim para destravar — e trava menos
- Chegar ao objetivo: é a primeira vez que o seu robô faz algo sozinho, de ponta a ponta
- Onde mora a decisão: é arquitetura, e é o que você vai defender na arguição
- Medir sem o robô: a demonstração do fim do semestre não pode depender de sorte
- As datas de cada etapa estão na página e em "Gates de cada TP" — consulte, não decore

Como a aula funciona hoje

- Primeira metade: clínica. Eu circulo, vocês põem o Nav2 para navegar
- A ferramenta de hoje diz ONDE OLHAR — ela não corrige e não valida nada
- Segunda metade: o item 2 começa, e ele tem duas entregas, não uma
- A árvore decide sem você; o bag prova sem você
- Se o tempo apertar, o bloco do bag cede: ele está inteiro na página, com os comandos

Pegar o material de hoje

```
# -- uma vez por terminal ------------------
export PB_USER=seu-usuario-github
export PB_DIR="$HOME/projeto-pb-$PB_USER"
export PB_WS="$PB_DIR/ros2_ws"

cd /tmp && rm -rf pb-a16 && \
  git clone --depth 1 https://github.com/Prof-Dacio-INFNET/PBRoboticos_prof_dacio.git pb-a16 && \
  rm -rf "$PB_DIR/aula16-clinica" "$PB_DIR/aula16-arvore-bag" && \
  cp -r pb-a16/exemplos/aula16-clinica "$PB_DIR/" && \
  cp -r pb-a16/exemplos/aula16-arvore-bag "$PB_DIR/"
```

Nenhum dos dois precisa de colcon build hoje: Python puro. E repare na cadeia — o rm só roda se o clone der certo, e o comentário fica ACIMA do bloco: dentro da cadeia ele a quebraria.

BLOCO 1

A clínica: onde olhar, em dez segundos

Com o seu Nav2 no ar, num terminal ao lado

```
cd "$PB_DIR/aula16-clinica"
python3 diagnosticar_nav2.py
python3 diagnosticar_nav2.py --rapido    # ambiente, nos, ciclo de vida, /cmd_vel
```

Dez verificações na ordem em que compensa olhar: ambiente, nós no ar, ciclo de vida, TF, pose inicial, /cmd_vel, a cadeia até o /cmd_vel_nav, as ações, os costmaps e o mapa.

achei. nao achei. olhe.

São os três estados, e nenhum deles é um veredito. "achei" não quer dizer certo; "nao achei" não quer dizer errado. Nos dois casos quer dizer: vá olhar isso. A ferramenta não corrige, não valida e não antecipa nota.

O que ele NÃO alcança — e é o que os gates cobram

- Se o robô realmente anda, e não só se os nós estão no ar
- Se o mapa publicado é do SEU cenário, e não o do exemplo
- Se a rota faz sentido na sua planta
- Se o relatório responde ao que foi pedido
- Lista vazia não é "pronto": é "esgotei o que um script enxerga"

Os quatro casos que respondem por quase tudo

- Gerenciador parado em Configuring: um nome na lista sem nó no ar — o primeiro inativo
- bt_navigator ativo e o goal não anda: falta map -> odom, que é a pose inicial
- Robô truncado ou andando sozinho: /cmd_vel com DOIS publicadores
- Robô parado e ninguém reclama: a cadeia /cmd_vel_nav -> velocity_smoother -> /cmd_vel

Os três comandos que separam os casos

```
ros2 lifecycle get /amcl             # active [3]
ros2 lifecycle get /bt_navigator     # active [3]
ros2 topic info /cmd_vel --verbose   # UM publicador
```

O diagnóstico roda estes e mais sete, e já interpreta. Mas vale saber os três de cor: "o Nav2 não navega" quase sempre é um deles respondendo outra coisa.

Nenhum dos quatro produz mensagem de erro.

É a mesma família do rviz:=true da semana passada. Por isso a primeira coisa a fazer não é ler o log: é perguntar ao sistema o que ele tem.

BLOCO 2

O robô indo sozinho, e depois a rota

O robô indo sozinho até um objetivo

```
# pelo RViz2: 2D Goal Pose (publica em /goal_pose)
# por codigo, com um ponto so:
ros2 run aula15_nav2 rota --ros-args -p rota:="[9.0, 2.0, 1.57]"
```

No SEU projeto o objetivo é um ponto do SEU mapa — escolhido sobre o costmap, não sobre a planta. E grave 20 s de vídeo: o robô chegando é a única prova que viaja. Print de plano desenhado é planejamento, não navegação.

Depois: a rota, e os dois costmaps explicados

- Três pontos ou mais, cumpridos em sequência — o rota.py com os SEUS pontos
- E os dois costmaps configurados e EXPLICADOS: a explicação vale tanto quanto a rota
- Global: o mapa inteiro com inflação, e é sobre ele que o NavFn planeja
- Local: janela rolante de poucos metros, do laser, e é sobre ela que o DWB decide
- No seu projeto: escreva isso com os SEUS números — janela, resolução, raio, inflação

Goal recusado não é goal que falhou.

Ponto em célula ocupada, ou a menos de robot_radius de uma parede, faz o servidor nem tentar. Escolha os pontos olhando o costmap global no RViz2, não o mapa cru. O robot_radius decide se a passagem abre; a inflação decide só quanto ela custa.

BLOCO 3

A árvore, e o que o Humble não tem

Os XML prontos, e os dois do exemplo

```
ls /opt/ros/humble/share/nav2_bt_navigator/behavior_trees/
cd "$PB_DIR/aula16-arvore-bag" && ls arvores/

# padrao_comentada.xml  -- a oficial, comentada bloco a bloco
# com_espera.xml        -- a variante com a espera de 2 s na recuperacao
```

O bt_navigator executa um XML. Copie para o SEU pacote e aponte pelo parâmetro do bt_navigator, ou pelo campo behavior_tree do goal — que o rota.py já sabe preencher, com arvore:=.

Não existe, no Humble, nó de árvore que meça distância a obstáculo.

Conferido na lista oficial de nós registrados: há condição para bateria baixa, para objetivo trocado, para caminho inválido, para tempo decorrido — e nenhuma para "obstáculo a menos de 0,5 m". A árvore sozinha NÃO cumpre o enunciado.

Então quem diz que há obstáculo é você, por um de dois caminhos

- Completo: um nó de condição próprio, em C++, plugin do nav2_behavior_tree...
- ...composto com o Wait de 2 s dentro de um ReactiveFallback
- De cliente: o vigia.py assina o /scan, cancela o goal, espera 2 s e reenvia
- O segundo é a Aula 8 de novo — cancelamento de ação — e não compila C++
- Os dois passam. O que não passa é não dizer qual você usou, e por quê

Duas armadilhas que não dizem o nome

- O wait_duration do nó Wait é INTEIRO: escrever 2.0 faz a árvore não carregar
- E o bt_navigator falha ao ativar sem apontar a linha
- Comentário de XML não pode conter dois traços seguidos
- Toda opção do ROS 2 começa com dois traços: colar --ros-args num <!-- --> ...
- ...faz o parser recusar o ARQUIVO INTEIRO, com mensagem que não fala de traços

BLOCO 4

A sessão gravada, e a figura

O caminho, e ele não precisa de biblioteca nova

```
ros2 bag record -o sessao-tp4 /odom /amcl_pose /scan /cmd_vel /tf /tf_static

# depois, com TUDO desligado:
ros2 bag play sessao-tp4 --clock
ros2 launch aula16_arvore analise.launch.py saida:=sessao.csv
python3 medir_sessao.py sessao.csv
```

O /tf_static sai UMA vez, no instante em que o sistema sobe. Fora da lista do record, a reprodução é um laser flutuando sem corpo — e você descobre depois, quando não há mais como regravar.

A régua antes do robô: a sessão sintética

```
cd "$PB_DIR/aula16-arvore-bag"
python3 simular_sessao.py                       # gera sessao.csv
python3 medir_sessao.py                         # a tabela
python3 plotar_trajetoria.py --saida trajetoria.png
```

Determinística, com dois eventos de obstáculo. Dá para fechar o gráfico e a tabela hoje, sem robô nenhum — e usar a MESMA régua no bag de vocês na semana que vem.

A tabela, e a primeira linha é a aula

```
DISTANCIA PERCORRIDA -- tres contas, tres numeros
  integral de |cmd_v| dt   :  25.52 m   <- o que foi mandado andar
  soma dos passos do /odom :  26.29 m   <- +3.0% (a deriva de escala)
  soma dos passos do /amcl :  49.03 m   <- +92.1%: o tremor virou caminho

INTERRUPCOES -- 2 evento(s); o portao viu no minimo 0.402 m
  1   t=12.30 s   parado 2.00 s   espera 2.00 s   2 s cumpridos: sim
  2   t=51.80 s   parado 2.10 s   espera 2.10 s   2 s cumpridos: sim
```

O robô andou 25 m e a trajetória do /amcl_pose tem 49. Não é defeito do AMCL: é a conta que está errada.

Somar passos de uma estimativa que treme soma o tremor.

Noventa e dois por cento de inflação saem de um ruído de poucos centímetros. Quem citar "distância percorrida" a partir do /amcl_pose está citando ruído — e esta é a terceira vez no semestre que uma métrica mede o instrumento, não o mundo.

As outras duas leituras da tabela

- O vão entre as duas trajetórias é 0,75 m, e ele tem nome
- Não é "erro do AMCL": é o que a transformada map -> odom corrige
- É a deriva acumulada da odometria — escreva assim no relatório
- E as duas esperas são 2,00 s e 2,10 s, diferentes de propósito
- Tolerância declarada antes de medir: 2,00 s ± 0,25 s. Como na Aula 9

O que levar para o seu projeto esta semana

- Um objetivo cumprido sem intervenção, com o vídeo de 20 s — é o que prova que anda
- Uma rota de três pontos do SEU mapa, e os dois parágrafos sobre os seus costmaps
- E a sua escolha de caminho para a interrupção de 0,5 m, trazida por escrito
- Verificação: o diagnóstico sem pendências, action list, topic info, tf2_echo
- A árvore vai para o SEU pacote, e o caminho apontado tem de existir no share instalado

Desafio — medir a sessão antes de ter uma

- Rode a régua no CSV sintético e leia a tabela inteira
- Depois mude UMA coisa: --sem-eventos, uma espera de 1,2 s, outra semente
- Qual coluna da tabela denuncia a mudança? E qual fica igual?
- Quem responder isso já sabe o que procurar no próprio bag na semana que vem

Na próxima: a sua escolha, funcionando ou travada.

Quem for de C++ traz o plugin compilando; quem for de cliente traz o vigia rodando. E tragam o bag. Começam também os itens 3 e 4, e neles a parte que não comprime são os 500 frames rotulados: comecem a salvar frames esta semana. As datas estão todas na página da aula e em "Gates de cada TP".